

IAM-XAI: Explainable Attack Path Risk Assessment for Cloud Identity Configurations

Alan Chris Dsilva*, Anna Mariya Jose, Bharathan K
and Julian Benedit P

Department of Computer Science and Engineering,
CHRIST (Deemed to be University), Bengaluru, India

* alanchrisdisilva@gmail.com

annamariya2605@gmail.com, bytexbharathan@gmail.com,
julian.p@christuniversity.in

Abstract

Misconfigured identity and access management (IAM) is a leading cause of cloud compromise. Chains of individually harmless permissions, such as passing a role to a compute service or assuming a role trusted by an external account, let an attacker escalate to administrator or reach sensitive data. Existing graph-based tools can enumerate such chains, but they rank them with fixed rules and rarely say why a path is dangerous or which single permission to remove. We present IAM-XAI, an end-to-end pipeline that parses AWS IAM configurations into an attack graph, enumerates privilege escalation paths, predicts the risk of each path with machine learning, explains every prediction with SHAP, and selects a small set of choke point permissions whose removal eliminates the most risk. Because labeled real-world IAM data is unavailable, we generate 3,000 randomized environments with 76,479 attack paths, labeled by an impact and likelihood risk oracle that uses information the model sees only partially. On environments never seen in training, a Random Forest reaches a macro F1 score of 0.881, close to the ceiling of 0.891 set by label ambiguity, while static rules reach 0.390. Pure machine learning misses up to 96.8 percent of severe paths for escalation primitives absent from training, and a small domain-knowledge floor removes these misses without loss of accuracy. The top SHAP feature names a true cause of risk for 95.3 percent of paths, and three choke point cuts remove 52.3 percent of ground-truth risk, close to the optimum of 52.7 percent. On 26 independently documented attacks and 6 benign controls, IAM-XAI detects 18 attacks with no false alarms.

1 Introduction

Public cloud platforms delegate almost all authorization decisions to identity and access management (IAM) policies. In Amazon Web Services (AWS), a principal’s effective power is the combination of the permission policies attached to it, the trust policies of the roles it may assume, and the permissions of any role it can *pass* to a service. These mechanisms compose: an identity that may only call `iam:PassRole` and `ec2:RunInstances` can launch an instance carrying an administrator role and thereby become administrator [13]. Over twenty such privilege-escalation methods are publicly cataloged [13], and deliberately vulnerable training environments reproduce them [27].

Graph-based analysis is the natural way to expose these compositions. Attack graphs have a long history in network security [32, 24, 25], tools such as Principal Mapper [23] and BloodHound [33] apply the idea to identity systems, and recent research detects multi-step IAM attacks by model checking [31] and repairs them automatically [17]. However, an environment with tens of identities and resources easily yields hundreds of paths. Security teams need three further things that path enumeration alone does not provide: (i) a *risk ranking* that reflects both the value of what is reached and how exposed the entry point is; (ii) an *explanation* of each ranking that an engineer can check; and (iii) a *minimal remediation*, meaning the few permissions whose removal breaks the most dangerous paths.

This paper presents IAM-XAI, a research pipeline that addresses all three for AWS-style IAM. It parses IAM configurations, builds an attack graph with four edge types, enumerates bounded cycle-free attack paths, and extracts a 27-field feature vector per path. Tree ensembles predict a four-level risk class, SHAP [20, 21] attributes each prediction to IAM characteristics, and a choke-point selector maps risk back to graph edges. The pipeline runs offline on configuration files and never touches a live cloud account.

A central difficulty is ground truth: there is no public corpus of real IAM configurations with expert risk labels. We therefore generate synthetic environments and label them with a documented risk oracle. An earlier version of this project labeled paths with rules over the model’s own input features and obtained a perfect but meaningless accuracy of 1.0. Following recommendations on pitfalls in security machine learning [1], we designed the evaluation so that such circularity and leakage are ruled out. The oracle uses hidden information, splits are grouped by environment, and results are checked against independently published attacks.

Our contributions are:

1. An end-to-end, reproducible pipeline from IAM configuration to risk prediction, per-path SHAP explanation and choke-point remediation, with a web

dashboard for interactive use.

2. A synthetic IAM environment generator and an impact $\times$ likelihood risk oracle whose labels are not a function of the model’s inputs, yielding a 76,479-path dataset with a measurable ambiguity ceiling.
3. A rigorous evaluation of six research questions: accuracy on unseen environments, generalization to unseen attack patterns, explanation faithfulness, remediation effectiveness, sensitivity to oracle assumptions, and detection of independently documented attacks.
4. Evidence that learned risk models fail on escalation primitives absent from training, and a *hybrid* model, a Random Forest with a small domain-knowledge floor, that removes those failures at no measurable cost.

Section 2 reviews related work, Section 3 describes the pipeline, Section 4 the dataset and oracle, Section 5 the evaluation protocol, and Section 6 the results. Sections 7 and 8 discuss limitations and conclude.

2 Background and Related Work

2.1 AWS IAM and privilege escalation

An AWS principal (user, group or role) receives permissions through identity-based policies of Allow and Deny statements over actions and resource ARNs, possibly guarded by conditions such as multi-factor authentication (MFA), source IP or an ExternalId. Roles additionally carry a *trust policy* naming who may assume them. The trusted party may be a principal in the same account, another account, a federated identity, a service, or the wildcard `”*`. An explicit Deny overrides any Allow. Privilege escalation arises when permissions compose. Examples include rewriting one’s own policy (`iam:CreatePolicyVersion`), attaching a managed policy, passing a privileged role to a new Lambda function or EC2 instance, or assuming a chain of roles [13].

2.2 Attack graphs

Attack graphs model how an adversary chains exploits or privileges toward a goal. Sheyner et al. [32] generated them with model checking, and MulVAL [24] and later logical attack graphs [25] made generation scalable with Datalog rules. Minimal cut sets of attack graphs identify the smallest set of measures that blocks every attack [18]. We apply the same idea to IAM permission edges, but weight paths by predicted risk instead of treating them uniformly.

2.3 Cloud IAM analysis

Formal verification. Zelkova [2] encodes AWS policies in SMT to answer questions such as “is this bucket public?”, and stratified abstraction [3] scales the approach to the engine behind IAM Access Analyzer. These methods reason exactly about one policy at a time, but they do not compose permissions into multi-hop escalation paths. Shevrin and Margalit [31] close that gap with bounded model checking: they turn an AWS IAM configuration into a state machine and find multi-step attacks with a formal guarantee. Their output is whether an attack exists, not how risky it is or which permission to remove.

Repair. IAMPERE [17] is the closest work to our remediation step. A graph neural network proposes a patch, and a MaxSAT solver shrinks it to a near-minimal change that provably removes every detected escalation. IAMPERE removes all escalations regardless of their consequence and does not explain its choices. IAM-XAI instead prioritizes cuts by predicted risk under a budget, and justifies each cut with feature attributions.

Practitioner tools. Principal Mapper [23] graphs AWS principals and reports which can escalate to administrator. Cloudsplaining [29] flags risky actions in policies, and BloodHound [33] popularized attack-path graphs for Active Directory. These tools apply fixed rules and produce binary findings.

Identity attack graphs with learning and cuts. Heat-ray [9] combined identity attack graphs, machine learning and edge-cut optimization to defend Windows domains, and Guo et al. [14, 15] compute budgeted edge blocking on Active Directory graphs. Choke-point selection is also the IAM counterpart of minimum-cost network hardening [34]. Graph neural networks have recently been trained on synthetic environment graphs to predict attack paths [12].

Positioning. To our knowledge, no prior work combines attack-path enumeration, a learned per-path risk score, SHAP explanations and budgeted choke-point selection for cloud IAM. We make no claim to the soundness guarantees of the model-checking and SMT approaches: IAM-XAI is a risk *prioritization* and explanation layer that could sit on top of such a sound path finder.

2.4 Machine learning and explainability in security

SHAP [20] attributes a model’s output to its input features using Shapley values, and TreeExplainer [21] computes them exactly and efficiently for tree ensembles such as Random Forests [5] and XGBoost [7]. LIME [28] is a model-agnostic alternative. Surveys of explainable AI for cybersecurity [6] note that explanations are rarely evaluated quantitatively. We therefore measure both whether explanations recover known causes and whether they are faithful to the model, using feature

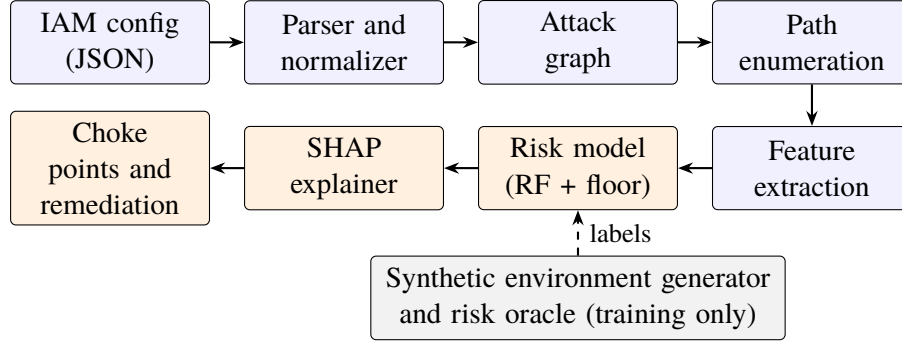


Figure 1: IAM-XAI pipeline, read left to right along the top row and right to left along the bottom row. Blue stages are deterministic graph analysis; orange stages are learned and explanatory. The generator and oracle are used only to build training and evaluation data.

deletion [30, 16]. Arp et al. [1] catalog pitfalls in security machine learning, including data snooping, spurious correlations and inappropriate baselines. Our grouped splits, static-rule baseline and external benchmark address these directly.

3 System Design

Figure 1 shows the pipeline. Every stage is a library module with a command-line entry point, and all randomness is seeded.

3.1 Parsing and normalization

The parser accepts AWS-style IAM JSON (users, groups, roles with trust and permission policies, resources) in both AWS casing (Effect, Action, Statement) and lower case. It validates structure, expands single values into lists, and emits a deterministic normalized representation of entities, permission statements and trust relationships.

3.2 Attack graph

Nodes are users, groups, roles, resources and synthetic principals for external accounts and the trust wildcard. Directed edges have four types: CAN_ASSUME (a trust policy admits the source and the source holds sts:AssumeRole), CAN_PASS_ROLE, CAN_ACCESS (read actions on a resource) and CAN_MODIFY (write or policy-changing actions). Each edge records its actions, resource patterns, conditions and provenance, so that it can later be

Table 1: Per-path feature vector (27 fields)

Group	Features
Numeric (12)	path length, role/user/resource count, action count, wildcard-action count, wildcard-resource count, broad-permission edges, conditional edges, condition keys, distinct services, classification tag
Binary (13)	assume role, pass role, wildcard action, wildcard resource, policy modification, external trust, cross-account, wildcard principal, sensitive target, admin permission, write access, has conditions, target privileged
Categorical (2)	target type, target service

mapped to the exact policy statement that creates it. Explicit Deny statements remove matching edges. Action- and resource-aware matching ensures a Deny on one bucket does not remove access to another. Conditional denials are conservatively ignored.

3.3 Attack path enumeration

A depth-first search from every identity enumerates simple (cycle-free) paths of at most six hops that end at a resource or a passable role. An adjacency index gives constant-time lookup of outgoing edges. Each path is serialized with its nodes, edges and hop count.

3.4 Feature extraction

A single shared function turns a path and the analyzer-visible scenario context into a feature row. Both the dataset generator and the dashboard call it, and a unit test checks that the two produce identical rows. It yields 27 fields (Table 1); after one-hot encoding the two categorical fields, the model sees 35 inputs. The context contains only what an analyst could read from the configuration. This includes resource types, any DataClassification tag, which roles hold administrator-equivalent (* or iam:*) permissions, and PassRole targets. It never contains the true sensitivity of the data.

3.5 Risk models

We train Logistic Regression (a linear, interpretable baseline), Random Forest (RF) [5] and XGBoost [7] to predict one of four classes: LOW, MEDIUM, HIGH or CRITICAL. From the primary model’s class probabilities $P(c \mid \mathbf{x})$ we also

derive an *expected risk*

$$\hat{r}(\mathbf{x}) = \sum_c P(c \mid \mathbf{x}) w_c, \quad w = (0.15, 0.40, 0.65, 0.85), \quad (1)$$

which, unlike the top-class probability, is low for a confident LOW prediction. It is used to weight paths during choke-point selection.

Hybrid model. Section 6.2 shows that a learned model cannot recognize escalation primitives it has never seen. The hybrid model therefore raises the RF’s label to a floor implied by well-known primitives: administrator-equivalent or policy-modification permissions $\Rightarrow$ at least HIGH; PassRole of a privileged role $\Rightarrow$ at least HIGH; any other PassRole $\Rightarrow$ at least MEDIUM. The floor never lowers a label.

3.6 SHAP explanations

For every prediction, TreeExplainer [21] computes the SHAP value ϕ_f of each feature f for the predicted class, so that the features’ contributions sum to the difference between that class’s probability and its average over the training data. The top six features by $|\phi_f|$ form the path’s explanation. Because many paths share a feature vector, SHAP values are computed once per distinct vector. This reduces the time to explain all 76k paths from roughly three hours to five minutes on a laptop.

3.7 Choke-point detection and remediation

A choke point is a graph edge, and hence a specific policy statement, whose removal breaks many high-risk paths. A fixed table maps each model feature to the edges that carry it. For example, pass_role maps to CAN_PASS_ROLE edges, and external_trust maps to CAN_ASSUME edges from external principals. Within a path p , edge e receives the score

$$s_p(e) = 0.1 + 2 \sum_{f \in \text{top}(p), f \mapsto e} |\phi_f|, \quad (2)$$

and the highest-scoring edge is the path’s own choke point. Across a scenario, each edge accumulates the expected risk $R(e) = \sum_{p \ni e} \hat{r}_p$ of Eq. (1) and the number of paths $n(e)$ it lies on, and is ranked by

$$S(e) = R(e) (1 + 0.3 (n(e) - 1)) + 0.5 B(e), \quad (3)$$

where $B(e)$ sums $s_p(e)$ from Eq. (2) over the paths whose own choke point is e . Ranking edges independently can waste cuts on paths that an earlier cut already broke. For a budget of k cuts we therefore also provide a *greedy* selector that solves

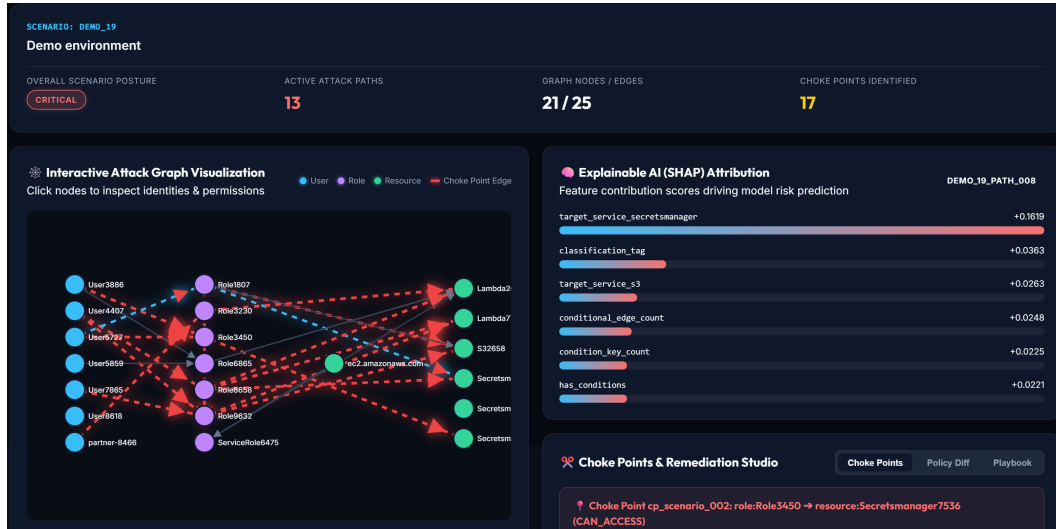


Figure 2: IAM-XAI dashboard on a generated environment (21 nodes, 25 edges, 13 attack paths). Top: overall posture and counts. Left: attack graph of users, roles and resources, with choke-point edges in dashed red. Right: SHAP attribution for the selected CRITICAL path, led by its Secrets Manager target and data-classification tag.

the weighted maximum-coverage problem: it repeatedly picks the edge on the largest remaining expected risk and then discards the paths it breaks. The remediation module converts a chosen edge into a minimal policy diff (removing the action, or the trust statement), re-runs path discovery on the patched configuration to verify that the paths are gone, and emits a human-readable playbook.

3.8 Dashboard

A local web dashboard runs the entire pipeline on an uploaded or bundled configuration (Figure 2). It shows the attack graph, per-path risk, SHAP factors, ranked choke points and one-click remediation with before-and-after verification.

4 Synthetic Dataset and Ground Truth

4.1 Environment generator

Each environment is built from its own seeded random stream, so generation is reproducible and parallel. It starts from a benign baseline of 3–20 users, 2–15 roles and 5–30 resources of six types: S3 buckets, DynamoDB tables, Lambda functions, EC2 instances, Secrets Manager secrets and KMS keys. Each resource has a hidden true data-classification tier: public, internal, confidential or restricted. Onto

this baseline, 0–3 of nine attack patterns are injected: wildcard action, wildcard resource, PassRole escalation, AssumeRole chain, external trust, cross-account trust, wildcard trust (Principal:"*"), policy modification and administrator wildcard. Mitigating conditions (MFA, source IP, ExternalId) and explicit Deny guardrails are added at random. Operators' tagging is modeled as imperfect: 85% of resources carry a DataClassification tag, and a few tags are one tier off.

4.2 Risk oracle

Labels must not be a rule over the model's own features, or learning becomes circular. The oracle scores each path as

$$\text{risk} = I \cdot (0.35 + 0.65 L), \quad (4)$$

where impact I is the maximum of the reached data's *true* tier impact (0.10, 0.30, 0.60 and 0.85 from public to restricted, +0.10 for write access), administrator power (0.95), policy-rewrite power (0.90), and PassRole of a role that is truly privileged (0.90) or not (0.35). Likelihood L is the exposure of the entry point (wildcard 1.0, external or cross-account 0.9, internal 0.6), decayed by 0.9 per additional hop and multiplied by a factor for each *specific* mitigating condition on the path (MFA 0.45, source IP 0.60, ExternalId 0.65). Cut points 0.20, 0.36 and 0.55 map the score of Eq. (4) to LOW, MEDIUM, HIGH and CRITICAL. The oracle also records a human-readable cause such as `restricted_data + external_trust`.

The model sees only partial evidence of the oracle's inputs: an imperfect tag instead of the true tier, whether *some* condition exists instead of which one, and so on. Identical feature vectors can therefore carry different labels, which makes the task non-trivial and gives it a measurable ceiling.

4.3 Dataset

We generate 3,000 environments. To stop dense environments from dominating, each is capped at 30 paths with class-balanced sampling. The result is 76,479 paths (25.5 per environment on average) with 2,068 distinct feature vectors. The classes are roughly balanced: LOW 20,049, MEDIUM 21,929, HIGH 19,598 and CRITICAL 14,903. The best accuracy any classifier can reach on these features, predicting the majority label of each distinct vector, is 0.891.

5 Evaluation Protocol

Splits. Data are split 70/15/15 into train, validation and test sets *grouped by environment*, so that no environment contributes paths to two splits. The random seed is fixed at 42.

Table 2: Test results on held-out environments (RQ1). Brackets give the 95% bootstrap confidence interval.

Model	Accuracy	Macro F1	ROC-AUC	Severe miss
Static rules	0.421	0.390 [0.371, 0.409]	–	13.5%
Logistic Regression	0.753	0.765 [0.753, 0.776]	0.914	12.9%
Random Forest	0.879	0.881 [0.870, 0.891]	0.974	9.8%
XGBoost	0.882	0.884 [0.874, 0.894]	0.979	10.4%
Hybrid (RF + floor)	0.878	0.880 [0.869, 0.890]	–	9.8%
Label-ambiguity ceiling (accuracy): 0.891				

Tuning. Models are implemented with scikit-learn [26] and XGBoost. A small grid per model (Logistic Regression: C ; RF with 300 trees: maximum depth, minimum leaf size, class weighting; XGBoost with 300 rounds: depth, learning rate) is searched on validation macro F1. The chosen configuration is refit on train and validation data and evaluated once on test.

Metrics and statistics. We report accuracy, macro F1, one-vs-rest ROC-AUC and the *severe-miss rate*: the share of truly HIGH or CRITICAL paths rated LOW or MEDIUM, the error that matters most to a defender. 95% confidence intervals come from 1,000 bootstrap resamples of test *environments* [10]; model pairs are compared with McNemar’s test [22, 8]; grouped 5-fold cross-validation measures stability.

Baseline. A static rule set assigns LOW to read-only access, MEDIUM to wildcards, HIGH to PassRole, role chains, long paths or sensitive targets, and CRITICAL to administrator, external or cross-account trust, policy modification, or a wildcard reaching a sensitive target. It represents conventional rule-based scoring.

Research questions. RQ1: How accurately is risk predicted on unseen environments? RQ2: Are attack patterns never seen in training recognized? RQ3: Are SHAP explanations correct and faithful? RQ4: Does cutting IAM-XAI’s choke points remove real risk? RQ5: Do conclusions depend on the oracle’s assumptions? RQ6: Are independently documented attacks detected?

6 Results

6.1 RQ1: Risk prediction on unseen environments

Table 2 shows that both tree ensembles come within about 0.01 of the 0.891 ceiling, so they have learned almost everything the features reveal about the oracle. Grouped 5-fold cross-validation agrees (macro F1: RF 0.884 ± 0.004 , XGBoost 0.889 ± 0.003 ,

True	LOW	88% (2717)	10% (308)	1% (42)	0% (6)
	MEDIUM	5% (161)	91% (3056)	3% (115)	1% (24)
	HIGH	3% (81)	10% (297)	81% (2347)	5% (156)
	CRITICAL	1% (28)	4% (96)	4% (86)	91% (2006)
		LOW	MEDIUM	HIGH	CRITICAL
		Predicted			

Figure 3: Random Forest confusion matrix on the test environments. Most errors are between adjacent risk levels.

Logistic Regression 0.754 ± 0.003). XGBoost’s advantage over RF is statistically detectable (McNemar $p = 0.005$; 93 vs. 58 discordant paths) but smaller than 0.01. We keep RF as the primary model because it has the lowest severe-miss rate. Logistic Regression trails by 0.12 macro F1, which indicates that risk depends on feature interactions, for instance that a sensitive target matters much more when the entry point is external. The static rules reach only 0.390 macro F1. They are too coarse, because a single wildcard or PassRole flag does not determine risk once exposure and mitigating conditions are accounted for. On the 11,526 test paths the hybrid floor turns 9 correct RF predictions into over-estimates and corrects none (McNemar $p = 0.004$), an accuracy cost of 0.001. Figure 3 shows that most RF errors fall between adjacent classes.

6.2 RQ2: Attack patterns never seen in training

For each of the nine attack patterns we train the RF only on environments *without* that pattern and test it on held-out paths that exercise it. Figure 4 shows the severe-miss rates. *Structural* patterns transfer well, because they combine features the model has seen in other contexts: assume-role chains, wildcard actions and resources, and external or cross-account trust (severe-miss 1.3–14.3%). Escalation *primitives* do not. With no PassRole example in training, the RF rates 96.8% of severe PassRole paths LOW or MEDIUM, and it misses 57.9% of administrator-wildcard paths. A learned model cannot infer that a permission it has never seen

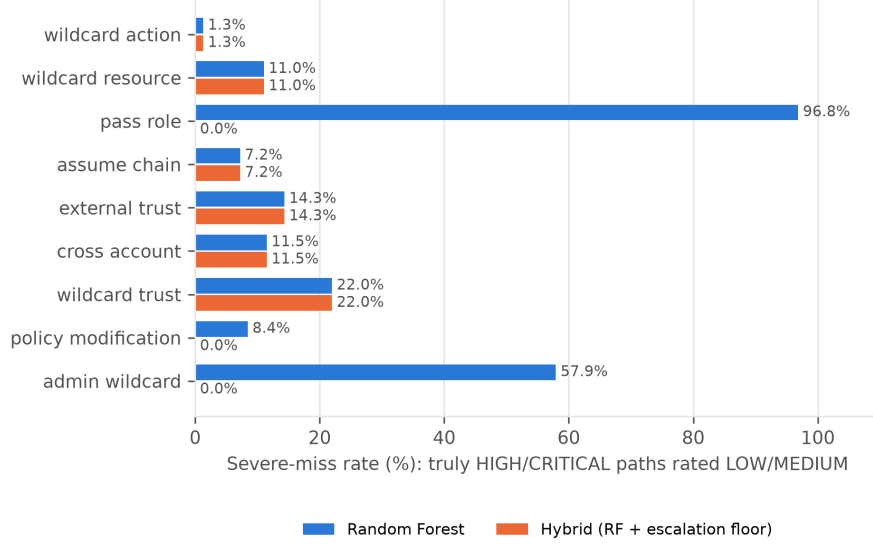


Figure 4: Leave-one-pattern-out severe-miss rate (RQ2). The escalation floor eliminates misses on the three privilege-escalation primitives.

grants administrator power. The hybrid floor encodes this domain knowledge and brings the severe-miss rate for PassRole, administrator wildcard and policy modification to 0%, while leaving in-distribution accuracy unchanged (RQ1). Wildcard trust (Principal:"*") remains the hardest structural pattern at 22.0%; unlike the primitives above, it is not covered by the floor and is a candidate for a further domain rule.

6.3 RQ3: Correctness and faithfulness of explanations

We sample 3,000 test paths and ask two questions. *Correctness*: do the top SHAP features name the oracle’s true drivers of risk? Each cause term, such as restricted_data or external_trust, is mapped to the feature group that can reveal it. *Faithfulness*: do the features SHAP ranks highest actually drive the model? We reset the top- k features to their training medians and measure the drop in the predicted class’s probability [30]. The baselines are a single global ranking (mean $|\phi|$ over all evaluated paths) and a random ranking.

Table 3 shows that the top SHAP feature names a true cause for 95.3% of paths. The global ranking does almost as well at top-1, because the target’s data classification is the dominant cause for most paths. The per-path explanations pull ahead where it matters. On paths with several causes they recover 77.0% of the true causes in their top three, against 70.6% for the global ranking. Removing SHAP’s top three features lowers the model’s confidence by 0.537, against 0.427 for the global rank-

Table 3: Explanation correctness and faithfulness (RQ3, 3,000 test paths)

Ranking	True cause recovered			Probability drop	
	Top 1	Top 3	Top 3, multi-cause	$k = 1$	$k = 3$
SHAP (per path)	95.3%	90.5%	77.0%	0.387	0.537
Global importance	95.0%	84.5%	70.6%	0.299	0.427
Random	29.9%	52.8%	38.4%	0.021	0.066

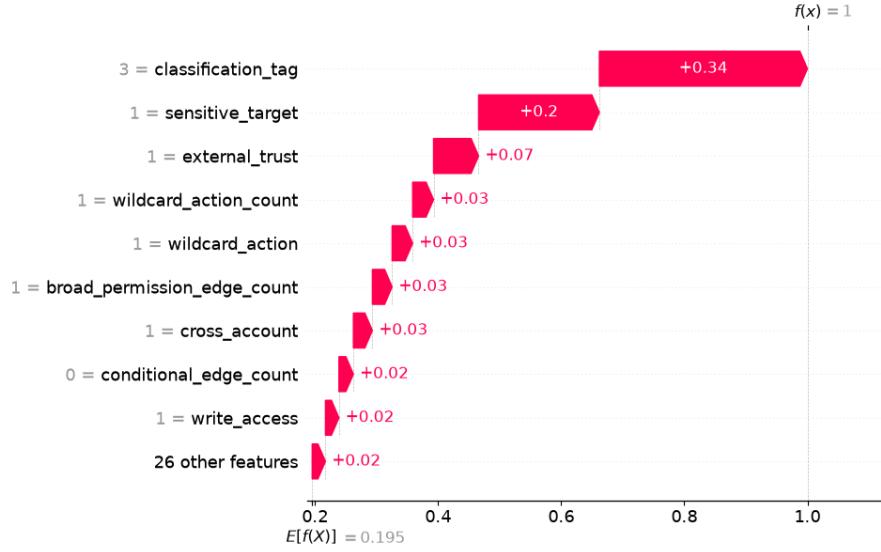


Figure 5: SHAP waterfall for a path predicted CRITICAL: a principal from an external account reaches a restricted resource with a wildcard action.

ing and 0.066 for a random one. The explanations therefore reflect what the model actually uses. Figure 5 shows a typical CRITICAL path. The restricted-data tag and sensitive target dominate, and external trust, a wildcard action and cross-account access add smaller contributions.

6.4 RQ4: Choke-point remediation

We generate 200 fresh environments with a seed never used in training (6,979 paths). We cut k edges chosen by each strategy and measure the share of *ground-truth oracle* risk removed. The strategies are a random edge, the most-shared edge (pure graph topology), IAM-XAI’s ranking $S(e)$ of Eq. (3), the same ranking without the SHAP term (ablation), the greedy selector, and an upper bound that runs greedy with the hidden oracle scores.

Table 4: Ground-truth risk removed by cutting k edges (RQ4, 200 unseen environments)

Strategy	Risk, $k = 1$	Risk, $k = 3$	Severe paths, $k = 3$
Random edge	7.2%	20.0%	20.8%
Most-shared edge	24.6%	43.9%	43.9%
IAM-XAI ranked	25.8%	46.6%	50.1%
ranked, no SHAP	26.0%	46.7%	49.5%
IAM-XAI greedy	26.4%	52.3%	59.7%
Oracle upper bound	26.7%	52.7%	61.3%

Table 5: Macro F1 under perturbed oracles (RQ5, 1,500 environments each)

Variant	Ceiling	Rules	LR	RF	Hybrid
Default	0.890	0.378	0.755	0.875	0.875
Label noise 5%	0.848	0.370	0.730	0.834	0.834
Label noise 10%	0.807	0.366	0.705	0.788	0.788
Weights perturbed (a)	0.897	0.378	0.775	0.886	0.886
Weights perturbed (b)	0.912	0.448	0.796	0.918	0.918
Weights perturbed (c)	0.888	0.389	0.747	0.871	0.871
Cut points -10%	0.878	0.388	0.770	0.863	0.863
Cut points $+10\%$	0.920	0.453	0.769	0.905	0.905

With three cuts, greedy selection removes 52.3% of real risk and 59.7% of severe paths (Table 4). This is 99% of the oracle optimum, and 7.3–9.7 percentage points more than the graph-only baseline (95% paired bootstrap confidence interval). The ablation clarifies where the gain comes from. Removing the SHAP term from the ranking changes almost nothing, so the improvement over pure topology comes from *machine-learned risk weighting*, and the gain of greedy over ranked selection comes from avoiding redundant cuts. SHAP’s role in remediation is to explain *why* an edge matters to the engineer who must approve its removal, not to select it. We report this as a finding, not a weakness: it separates the value of prediction from the value of explanation.

6.5 RQ5: Sensitivity to oracle assumptions

Because our labels come from an oracle we designed, we check whether the conclusions depend on its specific numbers. Each variant regenerates 1,500 environments with injected label noise, randomly perturbed oracle weights ($\pm 20\%$) or cut points shifted by $\pm 10\%$, and retrains all models (Table 5). In every variant the ordering

Table 6: External benchmark (RQ6)

Method	Attacks detected	False alarms (benign controls)
Random Forest	18/26 (69%)	0/6
Hybrid	18/26 (69%)	0/6
Static rules	18/26 (69%)	0/6

RF > Logistic Regression $\gg$ static rules holds. RF macro F1 stays within 0.02 of the variant’s accuracy ceiling, and the hybrid’s severe-miss rate stays between 7.2% and 12.5%. Label noise lowers the ceiling and every model with it, as expected.

6.6 RQ6: Independently documented attacks

To check that IAM-XAI is not merely learning its own generator, we encode 32 hand-written configurations and run them through the unchanged pipeline. Twenty-one are the IAM privilege-escalation methods cataloged by Rhino Security Labs [13]. Five are modeled on CloudGoat scenarios [27] and well-known trust misconfigurations, for example cross-account trust without an ExternalId. Six are benign controls. An attack counts as detected if any of its paths is rated HIGH or CRITICAL.

Attack paths were found for all 26 attack scenarios, and 18 were rated HIGH or CRITICAL with no false alarm on the benign controls (Table 6). The detection counts are equal across methods, but the learned models grade more precisely. They rate every PassRole-based attack (R03, R15, R16, R18, R20, R21 and CloudGoat-style C02, C03) CRITICAL where the rules say HIGH. They rate a public-trust role reading restricted data (C05) CRITICAL, and a write blocked by an explicit Deny (N06) LOW where the rules say MEDIUM. The eight misses share one cause: the attack graph does not yet model their escalation mechanism. These mechanisms are taking over another user’s credentials (iam:CreateAccessKey, CreateLoginProfile, UpdateLoginProfile), joining a group (iam:AddUserToGroup), rewriting the policy of a role the attacker can already assume (AttachRolePolicy, PutRolePolicy), and injecting code into a privileged service (lambda:UpdateFunctionCode, glue:UpdateDevEndpoint). We deliberately did not patch the graph after seeing these results, to avoid tuning on the benchmark. They define the next set of edge types to add.

7 Discussion and Threats to Validity

What the results mean. On the features an analyzer can extract, a tree ensemble approximates a principled impact $\times$ likelihood risk model almost perfectly, and it far

outperforms fixed rules. Explanations name true causes and are faithful to the model. Risk-weighted choke-point selection approaches the optimum. The most practically important result is RQ2. A purely data-driven risk model is blind to escalation primitives outside its training distribution, and a few lines of domain knowledge fix this at no measurable cost. We recommend that learned IAM risk scoring always be paired with such a floor.

Synthetic ground truth. The models learn our oracle, not expert judgment about real incidents. We mitigate this in four ways: the oracle follows the standard risk = impact $\times$ likelihood decomposition of NIST SP 800-30 [19] rather than a severity score such as CVSS [11], it uses information the model sees only partially, conclusions hold under perturbed weights, noise and cut points (RQ5), and detection is checked against independently published attacks (RQ6). A study with expert-labeled real configurations remains necessary.

Low-cardinality features. 98.2% of test feature vectors also occur in training. Grouped splits prevent environment-level leakage, and the ceiling bounds achievable accuracy, but RQ1 measures generalization to new environments more than to new feature combinations. RQ2 and RQ6 test the latter.

Graph coverage and scale. Credential takeover, group-membership changes and code injection are not modeled (RQ6). Path enumeration is exponential in dense graphs. Our synthetic environments are laptop-scale, and large enterprise accounts will need pruning or streaming enumeration.

Remediation feasibility. RQ4 assumes any edge can be cut without business impact. In practice some permissions are required, and a CAN_ASSUME patch currently revokes the whole trust statement instead of a single principal.

Explanation mapping. Mapping causes to features for RQ3 was defined by the authors. The random baseline shows that the mapping is not trivially satisfied, but other mappings could shift the absolute numbers.

8 Conclusion and Future Work

IAM-XAI turns an IAM configuration into ranked, explained and remediable privilege-escalation paths. On 3,000 synthetic environments with non-circular ground truth, a Random Forest predicts path risk at 0.881 macro F1, near the 0.891 ceiling and far above static rules (0.390). SHAP explanations recover true causes for 95% of paths and are faithful to the model. Three greedily chosen cuts remove 52.3% of real risk, 99% of the optimum. A domain-knowledge floor removes the blind spot of learned models on unseen escalation primitives. On 26 published attacks the pipeline detects 18 with no false alarms, and every miss traces to a known, documented gap in graph coverage.

Future work will (i) add edge types for credential takeover, group membership, role-policy rewriting and code injection, and evaluate them on a *fresh* set of techniques such as the 31 IAM Vulnerable scenarios [4]; (ii) validate risk labels against expert judgment on real, anonymized configurations; (iii) model the business cost of removing a permission in choke-point selection; and (iv) scale path enumeration to enterprise-size accounts. All code, data, trained models and results are reproducible from a single set of commands and are publicly available at <https://github.com/alan61503/IAM-XAI>.

Acknowledgments

The authors thank the Department of Computer Science and Engineering, CHRIST (Deemed to be University), for its support of this work.

References

- [1] Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., & Rieck, K. (2022). Dos and Don'ts of Machine Learning in Computer Security. In *Proceedings of the 31st USENIX Security Symposium* (pp. 3971–3988). USENIX Association.
- [2] Backes, J., Bolignano, P., Cook, B., Dodge, C., Gacek, A., Luckow, K., Rungta, N., Tkachuk, O., & Varming, C. (2018). Semantic-Based Automated Reasoning for AWS Access Policies Using SMT. In *Proceedings of the Formal Methods in Computer Aided Design (FMCAD)* (pp. 1–9). <https://doi.org/10.23919/FMCAD.2018.8602994>
- [3] Backes, J., Berrueco, U., Bray, T., Brim, D., Cook, B., Gacek, A., Jhala, R., Luckow, K., McLaughlin, S., Menon, M., Peebles, D., Pugalia, U., Rungta, N., Schlesinger, C., Schodde, A., Tanuku, A., Varming, C., & Viswanathan, D. (2020). Stratified Abstraction of Access Control Policies. In *Computer Aided Verification (CAV 2020)* (Lecture Notes in Computer Science, Vol. 12224) (pp. 165–176). Springer. https://doi.org/10.1007/978-3-030-53288-8_9
- [4] Bishop Fox. (n.d.). *IAM Vulnerable: An AWS IAM Privilege Escalation Playground*. <https://github.com/BishopFox/iam-vulnerable>. Accessed: Sep. 2026.
- [5] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>

- [6] Capuano, N., Fenza, G., Loia, V., & Stanzione, C. (2022). Explainable Artificial Intelligence in CyberSecurity: A Survey. *IEEE Access*, 10, 93575–93600. <https://doi.org/10.1109/ACCESS.2022.3204171>
- [7] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)* (pp. 785–794). <https://doi.org/10.1145/2939672.2939785>
- [8] Dietterich, T. G. (1998). Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms. *Neural Computation*, 10(7), 1895–1923. <https://doi.org/10.1162/089976698300017197>
- [9] Dunagan, J., Zheng, A. X., & Simon, D. R. (2009). Heat-ray: Combating Identity Snowball Attacks Using Machine Learning, Combinatorial Optimization and Attack Graphs. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)* (pp. 305–320). <https://doi.org/10.1145/1629575.1629605>
- [10] Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.
- [11] Forum of Incident Response and Security Teams (FIRST). (2019, June). *Common Vulnerability Scoring System v3.1: Specification Document*. <https://www.first.org/cvss/v3.1/specification-document>. Accessed: Sep. 2026.
- [12] François, M., Arduin, P.-E., & Merad, M. (2025). Physics-Informed Graph Neural Networks for Attack Path Prediction. *Journal of Cybersecurity and Privacy*, 5(2), 15. <https://doi.org/10.3390/jcp5020015>
- [13] Gietzen, S. (2018). *AWS IAM Privilege Escalation – Methods and Mitigation*. Rhino Security Labs blog, <https://rhinosecuritylabs.com/aws/aws-privilege-escalation-methods-mitigation/>. Accessed: Sep. 2026.
- [14] Guo, M., Li, J., Neumann, A., Neumann, F., & Nguyen, H. (2022). Practical Fixed-Parameter Algorithms for Defending Active Directory Style Attack Graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence* (pp. 9360–9367). <https://doi.org/10.1609/aaai.v36i9.21167>
- [15] Guo, M., Ward, M., Neumann, A., Neumann, F., & Nguyen, H. (2023). Scalable Edge Blocking Algorithms for Defending Active Directory Style Attack Graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence* (pp. 5649–5656). <https://doi.org/10.1609/aaai.v37i5.25701>

- [16] Hooker, S., Erhan, D., Kindermans, P.-J., & Kim, B. (2019). A Benchmark for Interpretability Methods in Deep Neural Networks. In *Advances in Neural Information Processing Systems 32 (NeurIPS)* (pp. 9737–9748).
- [17] Hu, Y., Wang, W., Khurshid, S., McMillan, K. L., & Tiwari, M. (2023). Fixing Privilege Escalations in Cloud Access Control with MaxSAT and Graph Neural Networks. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (pp. 104–115). <https://doi.org/10.1109/ASE56229.2023.00167>
- [18] Jha, S., Sheyner, O., & Wing, J. M. (2002). Two Formal Analyses of Attack Graphs. In *Proceedings of the 15th IEEE Computer Security Foundations Workshop (CSFW)* (pp. 49–63). <https://doi.org/10.1109/CSFW.2002.1021806>
- [19] Joint Task Force Transformation Initiative. (2012, September). *Guide for Conducting Risk Assessments* (NIST Special Publication 800-30 Rev. 1). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-30r1>
- [20] Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems 30 (NIPS)* (pp. 4765–4774).
- [21] Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., & Lee, S.-I. (2020). From Local Explanations to Global Understanding with Explainable AI for Trees. *Nature Machine Intelligence*, 2(1), 56–67. <https://doi.org/10.1038/s42256-019-0138-9>
- [22] McNemar, Q. (1947). Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika*, 12(2), 153–157. <https://doi.org/10.1007/BF02295996>
- [23] NCC Group, & Steringer, E. (2019). *Principal Mapper (PMapper): A Tool for Quickly Evaluating IAM Permissions in AWS*. <https://github.com/nccgroup/PMapper>. Accessed: Sep. 2026.
- [24] Ou, X., Govindavajhala, S., & Appel, A. W. (2005). MulVAL: A Logic-Based Network Security Analyzer. In *Proceedings of the 14th USENIX Security Symposium* (pp. 113–128). USENIX Association.
- [25] Ou, X., Boyer, W. F., & McQueen, M. A. (2006). A Scalable Approach to Attack Graph Generation. In *Proceedings of the 13th ACM Conference on Computer and Communications Security (CCS)* (pp. 336–345). <https://doi.org/10.1145/1180405.1180446>

- [26] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [27] Rhino Security Labs. (2019). *CloudGoat: “Vulnerable by Design” AWS Deployment Tool*. <https://github.com/RhinoSecurityLabs/cloudgoat>. Accessed: Sep. 2026.
- [28] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). “Why Should I Trust You?”: Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)* (pp. 1135–1144). <https://doi.org/10.1145/2939672.2939778>
- [29] Salesforce. (n.d.). *Cloudsplaining: AWS IAM Security Assessment Tool That Identifies Violations of Least Privilege*. <https://github.com/salesforce/cloudsplaining>. Accessed: Sep. 2026.
- [30] Samek, W., Binder, A., Montavon, G., Lapuschkin, S., & Müller, K.-R. (2017). Evaluating the Visualization of What a Deep Neural Network Has Learned. *IEEE Transactions on Neural Networks and Learning Systems*, 28(11), 2660–2673. <https://doi.org/10.1109/TNNLS.2016.2599820>
- [31] Shevrin, I., & Margalit, O. (2023). Detecting Multi-Step IAM Attacks in AWS Environments via Model Checking. In *Proceedings of the 32nd USENIX Security Symposium* (pp. 6025–6042). USENIX Association.
- [32] Sheyner, O., Haines, J., Jha, S., Lippmann, R., & Wing, J. M. (2002). Automated Generation and Analysis of Attack Graphs. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)* (pp. 273–284). <https://doi.org/10.1109/SECPRI.2002.1004377>
- [33] SpecterOps. (n.d.). *BloodHound: Graph-Based Analysis of Attack Paths in Identity and Access Management Systems*. <https://github.com/SpecterOps/BloodHound>. Accessed: Sep. 2026.
- [34] Wang, L., Noel, S., & Jajodia, S. (2006). Minimum-Cost Network Hardening Using Attack Graphs. *Computer Communications*, 29(18), 3812–3824. <https://doi.org/10.1016/j.comcom.2006.06.018>